Revision notes

Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity"
(Google DeepMind, ICLR 2026)

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Contents

1 Notation **1**

2 Why quantize, and what training looks like **2**

3 The quantizer (paper Eq. 1) **2**

4 The problem: the true gradient is zero **2**

5 The standard fix: the straight-through estimator (STE) **3**

6 The new method: ridge dequantization g (Eqs. 2–3) **3**

7 g as a normalization layer (Sec. 4) **4**

8 The backward pass: how δ reaches the gradient **4**

9 Efficient affine matrix multiplication (Theorem 1, Sec. 5) **5**

10 Sparsity as a special form of quantization (Sec. 3.4, A.1.5) **5**

11 Results from the paper **6**

12 Our implementation **6**

13 Weak points of the paper **7**

14 Viva questions and answers **7**

1 Notation

A *block* is a vector $x \in \mathbb{R}^N$ that shares one scale and one shift. Means and variances are population moments $(1/N)$:

$$\bar{x} = \tfrac{1}{N}\mathbf{1}^\top x, \qquad \mathrm{Var}_q = \tfrac{1}{N}\|q - \bar{q}\mathbf{1}\|^2, \qquad \mathrm{Cov}_{xq} = \tfrac{1}{N}(x - \bar{x}\mathbf{1})^\top(q - \bar{q}\mathbf{1}).$$

Throughout, $u = q - \bar{q}\mathbf{1}$ (centred integers), k is the bit width, and the integer grid is $\{0, 1, \ldots, 2^k - 1\}$.

2 Why quantize, and what training looks like

Motivation. A language model has billions of weights stored in 16 or 32 bits. Quantization stores each in $k \in \{4, 2, 1\}$ bits, plus one scale and shift per block. This cuts memory, memory traffic and arithmetic cost.

Weights vs. activations. *Weights* are learned parameters, fixed after training. *Activations* are the vectors flowing between layers for a given input; they change with every input. A linear layer computes $y = Wx$ with W weights and x activations. Both are quantized because the cost of a multiply scales with (weight bits)×(activation bits): 1-bit weights with 16-bit activations still need 16-bit additions.

Quantization-aware training ("fake quantization"). Training keeps the float weights x and updates them by gradient descent. The forward pass uses the quantized-then-reconstructed values, so the network learns to tolerate rounding. Reconstruction is done in float during training only so that autograd can run; it carries exactly the information of the integers q. After training only q and the per-block scalars are kept.

Where quantization is applied. At the input and weights of every linear layer (in a transformer: QKV and output projections, MLP up and down projections). Embeddings, LayerNorm, softmax and the attention products $QK^\top$, softmax $\cdot\, V$ stay in float in our runs.

3 The quantizer (paper Eq. 1)

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x}, \qquad f(x) = \frac{x - b_f \mathbf{1}}{s_f}, \quad s_f = \frac{x_{\max} - x_{\min}}{2^k - 1}, \quad b_f = x_{\min}.$$

Write $q = f(x) + \delta$ with rounding error $|\delta_i| \le \frac{1}{2}$. Then

$$\boxed{\hat{x} = s_f q + b_f \mathbf{1} = x + s_f \delta.}$$

Quantization is x plus a scaled rounding error. Everything else builds on this identity.

Worked example ($k = 2$). $x = (0.9, -0.3, 0.1, 0.6)$. Then $s_f = 1.2/3 = 0.4$, $b_f = -0.3$, $f(x) = (3, 0, 1, 2.25)$, $q = (3, 0, 1, 2)$, $\hat{x} = (0.9, -0.3, 0.1, 0.5)$. Squared error $\|\hat{x} - x\|^2 = 0.010$.

4 The problem: the true gradient is zero

By the chain rule, with s_f, b_f held fixed,

$$\frac{\partial \hat{x}}{\partial x} = s_f \cdot \frac{\partial q}{\partial f(x)} \cdot \frac{1}{s_f} = \frac{\partial q}{\partial f(x)}.$$

round is a step function: its derivative is 0 off the half-integers and undefined on them. So $\nabla_x L = 0$ almost everywhere. A small change of x does not change q; x and all earlier layers never move.

Caveat. s_f and b_f depend on x through $x_{\max}$ and $x_{\min}$, so the two extreme entries of each block do get a non-zero gradient. The paper's analysis (and ours) holds s_f, b_f fixed.

5 The standard fix: the straight-through estimator (STE)

Detach the rounding error:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\big(\text{round}(f(x)) - f(x)\big).$$

stopgrad is the identity in the forward pass and has zero derivative in the backward pass. Hence $\partial q/\partial f(x) = I$ and

$$\text{forward: } \hat{x} = x + s_f\delta, \qquad \text{backward: } \frac{\partial \hat{x}}{\partial x} = I, \quad \nabla_x L = \nabla_{\hat{x}} L.$$

(Bengio, Léonard, Courville, 2013.)

The blind spot. The forward pass is shifted by $s_f\delta$, but the Jacobian is I for every δ: the gradient is exactly what it would be with no quantization. Earlier layers receive no signal about how rounding hurts the loss. At 8 bits $s_f\delta$ is tiny. At 1–2 bits it is as large as the values themselves.

Paper's evidence (Fig. 1). (a) Small GPT on Tiny Shakespeare, A1W1: STE's loss climbs and oscillates around 5–8; the new method converges. (b) Gemma 3 1B, A1W1, SCQ128: under STE affine quantization gives nothing (linear 0.3399, affine 0.3397); with g it helps (0.3547 vs. 0.3751). The paper's reading: STE cannot learn the sensitive shift term, because its gradient ignores the rounding error.

6 The new method: ridge dequantization g (Eqs. 2–3)

Keep f and round. Replace $\hat{x} = s_f q + b_f\mathbf{1}$ by an affine map whose coefficients are *fitted to* x:

$$g(q) = s_g q + b_g\mathbf{1}, \qquad (s_g, b_g) = \arg\min_{s,b} \frac{1}{2N}\|sq + b\mathbf{1} - x\|^2 + \frac{\lambda}{2}s^2.$$

Claim. $s_g = \dfrac{\text{Cov}_{xq}}{\text{Var}_q + \lambda}, \quad b_g = \bar{x} - s_g\bar{q}.$

Proof. The objective $J(s,b)$ is a strictly convex quadratic for $\lambda > 0$. Setting $\partial J/\partial b = \frac{1}{N}\sum_i(sq_i + b - x_i) = 0$ gives $b = \bar{x} - s\bar{q}$. Substituting, the residual is $s\,u_i - (x_i - \bar{x})$, and $\partial J/\partial s = \frac{1}{N}\sum_i \big(su_i - (x_i - \bar{x})\big)u_i + \lambda s = s\,\text{Var}_q - \text{Cov}_{xq} + \lambda s = 0$. (Using $\sum_i u_i = 0$ to drop the b terms.) □

Equivalent centred form: $g(q) = s_g(q - \bar{q}\mathbf{1}) + \bar{x}\mathbf{1}$.

Role of λ (the "denoising knob", default 0.01 everywhere).

- $\lambda \to \infty$: $s_g \to 0$, $g \to \bar{x}\mathbf{1}$ (ignore the noisy q, output the mean).
- $\lambda = 0$: ordinary least squares. Undefined when $\text{Var}_q = 0$ (all entries round to the same integer); Table 13 shows $\lambda = 0$ gives NaN.
- Table 13 (A1W1, WMT EN–DE BLEU): $\lambda = 1$: 5.83; 0.01: 21.42; 10^{-4}: 20.08; 0: NaN.

δ is inside s_g. From $x = s_f(q - \delta) + b_f\mathbf{1}$,

$$\text{Cov}_{xq} = s_f\big(\text{Var}_q - \text{Cov}_{q\delta}\big) \quad \Longrightarrow \quad s_g = s_f\,\frac{\text{Var}_q - \text{Cov}_{q\delta}}{\text{Var}_q + \lambda}.$$

The residual is

$$r = x - g(q) = (s_f - s_g)\,u - s_f(\delta - \bar{\delta}\mathbf{1}),$$

i.e. the rounding error left over after the fit.

Is g a better reconstruction?

- With $\lambda = 0$, yes always: $\hat{x}$ is one particular affine function of q, and g is the least-squares best one, so $\|g(q) - x\| \le \|\hat{x} - x\|$.
- With $\lambda > 0$, usually but not always (the penalty can push it slightly off). Our check on 10^4 random blocks: g is worse in 153 blocks at 1 bit; median error ratio $g/\hat{x}$ is 0.21 at 1 bit and 0.97 at 4 bits.
- Worked example: $\bar{q} = 1.5$, $\mathrm{Var}_q = 1.25$, $\mathrm{Cov}_{xq} = 0.5125$, $s_g = 0.4067$, $b_g = -0.2851$, $g(q) = (0.935, -0.285, 0.122, 0.528)$, squared error 0.007 vs. 0.010.

The 1-bit case. With $q \in \{0, 1\}$, an affine map of q just picks two levels. Least squares ($\lambda = 0$) puts them at the *means of the two groups*. STE's $\hat{x}$ puts them at $x_{\min}$ and $x_{\max}$, which is too far out. This is why STE's activations grow too large at 1 bit.

Linear (symmetric) variant (Eq. 4). For zero-centred data use $g(q) = s_g q$ with $s_g = \overline{qx}/(\overline{q^2} + \lambda)$. For signs $q = \pm 1$: $s_g = \overline{|x|}/(1 + \lambda)$, close to the mean absolute value (the XNOR-Net scale). STE with $s_f = \max|x|$ uses levels $\pm \max|x|$.

Which variant where. Affine g for uncentred data such as activations (Sec. 3.3.1); linear g for roughly zero-centred data, e.g. ternary sparse weights.

7 g as a normalization layer (Sec. 4)

$$g(q) = \underbrace{\frac{\mathrm{Cov}_{xq}}{\sqrt{\mathrm{Var}_q + \lambda}}}_{\sim\gamma} \cdot \underbrace{\frac{q - \bar{q}\mathbf{1}}{\sqrt{\mathrm{Var}_q + \lambda}}}_{\sim(q-\mu)/\sigma} + \underbrace{\bar{x}\mathbf{1}}_{\sim\beta}.$$

This is LayerNorm applied to q, with λ in the role of ϵ, and with γ, β *computed* from x rather than learned. Cost is about one LayerNorm per block (RMSNorm for the linear variant). g has no learnable parameters.

8 The backward pass: how δ reaches the gradient

Both methods use the same surrogate for round: $\partial q/\partial x = I/s_f$. Only the step after round differs.

STE. $\partial\hat{x}/\partial q = s_f I$ for every δ, so the Jacobian is I.

With g: two paths. g depends on x through q and also directly (through $\bar{x}$ and Cov_{xq}):

$$\nabla_x L = \underbrace{\frac{1}{s_f}\Big(\frac{\partial g}{\partial q}\Big)^\top \nabla_g L}_{\text{through } q \text{ (surrogate)}} + \underbrace{\Big(\frac{\partial g}{\partial x}\Big)^\top_{q \text{ fixed}} \nabla_g L}_{\text{through } x \text{ (exact)}}.$$

Our derivation of the full Jacobian. With $\rho = s_g/s_f$, $c = 1/\big(s_f N(\mathrm{Var}_q + \lambda)\big)$ and

$$P = \frac{\mathbf{1}\mathbf{1}^\top}{N} + \frac{uu^\top}{N(\mathrm{Var}_q + \lambda)}, \qquad \boxed{J_g = \frac{dg}{dx} = P + \rho\,(I - P) + c\,u\,r^\top}, \qquad J_{\mathrm{STE}} = I.$$

- P is the exact (a.e.) Jacobian: with q fixed, $g = s_g u + \bar{x}\mathbf{1}$ is linear in x; $\partial\bar{x}/\partial x = \mathbf{1}^\top/N$ and $\partial s_g/\partial x = u^\top/(N(\mathrm{Var}_q + \lambda))$. It depends only on q, so it carries no information about δ.
- With $\lambda = 0$, P is the orthogonal projection onto $\mathrm{span}\{\mathbf{1}, u\}$ (since $\|u\|^2 = N\,\mathrm{Var}_q$ and $\mathbf{1} \perp u$). With $\lambda > 0$ the u direction is slightly shrunk.
- $\rho(I - P)$ and $c\,ur^\top$ pass through the surrogate. ρ depends on δ through s_g; r is the leftover rounding error. This is how δ enters the gradient.
- Sanity check: if $\rho = 1$ and $r = 0$, $J_g = I$, i.e. STE.
- Verified against autograd in `implementation/jacobian_check.py` (excluding the argmin and argmax entries, since s_f, b_f depend on them).

Sizes on a random block of 64.

	1 bit	2 bits	4 bits
$\rho = s_g/s_f$	0.29	0.87	0.99
share of upstream gradient in $c\,ur^\top$	0.5%	4.7%	0.3%
share in exact part P	14.8%	21.1%	3.2%

At 4 bits $J_g \approx I$: g behaves almost like STE, consistent with the tiny 4-bit gain. At 1 bit the surrogate path is scaled down to 0.29 and the exact path matters.

The paper's own analysis (A.4) derives only $\partial g/\partial q$, for the linear case. It misses the exact direct path P.

9 Efficient affine matrix multiplication (Theorem 1, Sec. 5)

Each row of X (one token) and each column of W (one output channel) has its own affine reconstruction $s(q - \bar{q}\mathbf{1}) + \bar{x}\mathbf{1}$. Then

$$\boxed{Y = (s_X s_W^\top) \odot \left(Q_X Q_W - n\,\bar{q}_X \bar{q}_W^\top\right) + n\,\bar{x}\,\bar{w}^\top.}$$

Proof sketch. Write $\hat{X} = D_X(Q_X - \bar{q}_X\mathbf{1}^\top) + \bar{x}\mathbf{1}^\top$ and $\hat{W} = (Q_W - \mathbf{1}\bar{q}_W^\top)D_W + \mathbf{1}\bar{w}^\top$. In $\hat{X}\hat{W}$ the cross terms vanish because centred rows and columns sum to zero: $(Q_X - \bar{q}_X\mathbf{1}^\top)\mathbf{1} = 0$ and $\mathbf{1}^\top(Q_W - \mathbf{1}\bar{q}_W^\top) = 0$. The constant term is $\bar{x}\mathbf{1}^\top\mathbf{1}\bar{w}^\top = n\bar{x}\bar{w}^\top$. The main term expands, using $Q_X\mathbf{1} = n\bar{q}_X$ and $\mathbf{1}^\top Q_W = n\bar{q}_W^\top$, to $Q_X Q_W - n\bar{q}_X\bar{q}_W^\top$. □

So an affine-quantized matmul is one *integer* matmul plus rank-1 corrections. At 1 bit the integer matmul is XNOR plus popcount. (Paper typo: Sec. 5 text writes $\bar{q}_X\bar{q}_X^\top$.)

Inference cost of g. For weights, s_g, b_g are computed once after training. For activations, x is new for every input, so each token needs its float $\bar{x}$ and Cov_{xq} at run time: an $O(N)$ pass, about a LayerNorm, cheap next to the $O(N \cdot \mathrm{width})$ matmul, but not counted in the energy proxy.

10 Sparsity as a special form of quantization (Sec. 3.4, A.1.5)

1. Threshold small values to zero (e.g. 2:4 structured: 2 non-zeros per group of 4). Sparsity error $\delta_S = \mathrm{threshold}(x) - x$, so $x_S = x + \delta_S$.

2. Quantize: $q = f(x_S) + \delta_Q$.
3. Reconstruct with g, fitted against the original *dense* x. So s_g, b_g adapt to the combined error $\delta_S + \delta_Q$, and the backward pass sees both.

1-bit weights with 2:4 sparsity give ternary weights $\{-1, 0, 1\}$. These are roughly centred, so the paper uses *linear* g; this also keeps zeros at zero (affine g would map 0 to $b_g \neq 0$).

Costs: 2:4 halves the multiplies (energy factor 0.5). Storage: 1:4 gives $(1 + 2)/4 = 0.75$ bits per element; 2:4 needs a 4-bit mask, $(2 + 4)/4 = 1.5$.

11 Results from the paper

Gemma 3 1B, affine SCQ128	STE	with g	gain
4 bits	0.4254	0.4274	+0.002
2 bits	0.3794	0.4027	+0.023
1 bit	0.3397	0.3751	+0.035

Score: next-token accuracy. Fewer bits, bigger gain. SCQ128 = sub-channel blocks of 128.

- A1.5W1.5 (ternary, $\log_2 3 \approx 1.58$ bits), linear channel-wise: STE NaN, g 0.3297.
- Table 6: Gemma 3 4B, A4W1, 2:4 sparse: 650 MB, 0.4517. Gemma 3 1B BF16: 1400 MB, 0.4494. Gemma 3 4B BF16: 0.4706.
- Table 7: ResNet-50 A4W4 76.45% vs. float 76.41%. Table 9: WMT EN–DE A4W4 29.71 vs. 29.49 BLEU.
- Energy proxy (A.1.6): sparsity × act. bits × weight bits × ops. BF16 = 256, A4W4 = 16, A1W1 = 1. An estimate, not a measurement. (Paper typo: "16 × 16 = 26".)

12 Our implementation

Data. Tiny Shakespeare: about 1.1M characters of Shakespeare play dialogue, character-level, vocabulary of 65 characters, first 90% train, last 10% validation. Loss is cross-entropy in nats per character; a uniform guess gives $\ln 65 \approx 4.17$.

Small toy. 4 blocks, 0.82M parameters, context 64, 3000 steps. Final validation loss, STE vs. g: 1 bit 2.123 vs. 2.071; 2 bits 1.812 vs. 1.790; 4 bits 1.592 vs. 1.597. Float 1.586. No run diverged.

Paper-size 1-bit run (`onebit/`). nanoGPT `train_shakespeare_char`: 6 layers, 6 heads, width 384, 10.7M parameters. Every linear layer in the blocks has 1-bit weights (one block per output row) and 1-bit inputs (one block per token), so $N = 384$ or 1536. We used batch 32, context 128, 1000–2000 steps (paper: 64, 256, 5000).

- Three settings: affine 1-bit, sign (linear) 1-bit, affine at 3× learning rate. In all three STE stalls near 2.6 while g keeps learning to about 2.35.
- STE's residual stream is about 340× too large after 1000 steps (617 vs. 1.8). Cause: STE's two levels sit at $x_{\min}, x_{\max}$; g's sit near the group means. LayerNorm hides the size from the loss, but each layer's update is swamped.
- At lr 3×10^{-3} STE's training loss turns upward after about step 75; g reaches 2.29.

- Not reproduced: the paper's loss explosion to 6–8. Likely because we trained on fewer tokens per step and fewer steps.

13 Weak points of the paper

1. "Without gradient estimation" (abstract, Table 8) is too strong: δ is detached, so round still uses the identity surrogate, as in STE. The new information about δ arrives *through* that surrogate path.
2. "For the first time" clashes with the abstract's own "well-understood mismatch". LSQ (Esser et al., 2019) already puts δ into the step-size gradient (not the upstream gradient).
3. A.4 derives only $\partial g/\partial q$ for the linear case; it misses the exact path P.
4. Effective bits per element ignore per-block scale and shift (dense 1-bit listed as 1.00 BPE, Table 5).
5. Single runs, no error bars. The 4-bit gain (+0.002) may be noise.
6. BitNet is run at A1W1, though the paper itself says BitNet's lowest activation precision is 8 bits.
7. Energy savings are a proxy; no hardware or speed numbers. The per-token float cost of g on activations is not counted.
8. No ablation of sparsity under STE vs. g.
9. Typos: $\bar{q}_X \bar{q}_X^\top$ in Sec. 5; $16 \times 16 = 26$ in A.1.6.

14 Viva questions and answers

Q1. Why can't we just backpropagate through the quantizer?

Answer. round is piecewise constant, so its derivative is 0 almost everywhere and undefined at half-integers. The chain rule gives $\nabla_x L = 0$, so no weight before the quantizer ever moves.

Q2. What exactly is STE, and why "straight-through"?

Answer. It writes $q = f(x) + \text{stopgrad}(\text{round}(f(x)) - f(x))$. Forward it rounds; backward $\partial q/\partial f = I$, so the gradient passes through the rounding step as if it were the identity.

Q3. What is wrong with STE if training still runs?

Answer. Its Jacobian is I for every δ, so the gradient is blind to the rounding error. Earlier layers cannot learn to be robust to rounding. Harmless at 8 bits; at 1–2 bits $s_f \delta$ is as large as the values, and STE diverges or stalls.

Q4. Does the new method avoid the surrogate gradient?

Answer. No. δ is still detached, so $\partial q/\partial f = I$ exactly as in STE. What changes is the step after rounding: g's Jacobian depends on δ (through s_g and the residual r) and adds an exact direct path P. So "without gradient estimation" is an overstatement.

Q5. Derive s_g and b_g.

Answer. Set the partial derivatives of $\frac{1}{2N}\|sq + b\mathbf{1} - x\|^2 + \frac{\lambda}{2}s^2$ to zero. ∂_b: $b = \bar{x} - s\bar{q}$. Substitute; ∂_s: $s(\text{Var}_q + \lambda) = \text{Cov}_{xq}$.

Q6. Why ridge and not plain least squares?

Answer. Two reasons. (1) Stability: if all entries round to the same integer, $\mathrm{Var}_q = 0$ and OLS divides by zero; $\lambda = 0$ gives NaN in Table 13. (2) Denoising: λ shrinks s_g toward 0 when q is a poor predictor of x, falling back to $\bar{x}$.

Q7. Why is only s_g penalised, not b_g?

Answer. b_g is the shift; penalising it would bias the output mean away from $\bar{x}$. The noise comes in through q, which only s_g multiplies.

Q8. What happens as $\lambda \to 0$ and $\lambda \to \infty$?

Answer. $\lambda \to 0$: OLS, best affine fit, unstable when $\mathrm{Var}_q \approx 0$. $\lambda \to \infty$: $s_g \to 0$, $g(q) \to \bar{x}\mathbf{1}$, all information in q discarded.

Q9. Is $g(q)$ always closer to x than $\hat{x}$?

Answer. For $\lambda = 0$, yes: $\hat{x}$ is one affine function of q and g is the best one. For $\lambda > 0$, not always: worse in 153 of 10^4 random 1-bit blocks in our check, but the median error ratio is 0.21 at 1 bit.

Q10. Where does δ appear in the backward pass?

Answer. In $J_g = P + \rho(I - P) + c\,ur^\top$. $\rho = s_g/s_f = (\mathrm{Var}_q - \mathrm{Cov}_{q\delta})/(\mathrm{Var}_q + \lambda)$ depends on δ, and $r = (s_f - s_g)u - s_f(\delta - \bar{\delta}\mathbf{1})$ is the leftover rounding error.

Q11. What is P?

Answer. The exact Jacobian of g with respect to its direct dependence on x (through $\bar{x}$ and Cov_{xq}), with q fixed. For $\lambda = 0$ it is the orthogonal projection onto $\mathrm{span}\{\mathbf{1}, u\}$. It needs no surrogate but carries no information about δ.

Q12. Isn't g just a rescaled STE?

Answer. Partly. The surrogate path is scaled by ρ rather than 1. But g adds the exact path P and the residual term $c\,ur^\top$, and ρ itself depends on δ. STE's slope never does.

Q13. Why is the 4-bit gain so small?

Answer. At 4 bits δ is small relative to the spread of q, so $\rho \approx 0.99$ and $r \approx 0$; $J_g \approx I$, i.e. almost STE. Also single runs, so +0.002 may be noise.

Q14. Why does affine quantization not help under STE?

Answer. The paper's explanation: the shift is a sensitive parameter, and STE's gradient ignores rounding error, so it cannot learn a useful shift. With g, $b_g = \bar{x} - s_g\bar{q}$ is fitted in closed form, and affine helps (0.3547 to 0.3751).

Q15. Explain the LayerNorm analogy.

Answer. $g(q) = \frac{\mathrm{Cov}_{xq}}{\sqrt{\mathrm{Var}_q + \lambda}} \cdot \frac{q - \bar{q}}{\sqrt{\mathrm{Var}_q + \lambda}} + \bar{x}$: normalise q, then rescale and shift with statistics of x. λ plays the role of ϵ. Cost is about one LayerNorm; the normalisation structure supplies the extra gradient path.

Q16. Does g add learnable parameters?

Answer. No. s_g and b_g are computed in closed form from each block; only λ is a fixed hyperparameter (0.01 everywhere).

Q17. State and prove Theorem 1.

Answer. See Section 9. Centred rows/columns sum to zero, so cross terms vanish; the result is one integer matmul $Q_X Q_W$ plus rank-1 corrections.

Q18. The training code multiplies float reconstructions. Where is the saving?

Answer. That is fake quantization, used so autograd can run. The reconstruction is affine with per-block scalars, so the scalars factor out of each dot product (Theorem 1). Deployed hardware does an integer matmul on q_X, q_W and a cheap per-block rescale.

Q19. Why quantize activations, not just weights?

Answer. Multiply cost scales with weight bits $\times$ activation bits. With float activations a 1-bit weight saves storage but the adds are still 16-bit. With both 1-bit the dot product is XNOR plus popcount. Activations also dominate memory traffic.

Q20. Do we compute s_g, b_g for activations too? At what cost?

Answer. Yes, per token, at every quantized layer (the paper uses affine g for activations). This needs the float x at run time: an $O(N)$ pass, about a LayerNorm. Weights' s_g, b_g are precomputed. The energy proxy ignores this cost.

Q21. How is sparsity handled?

Answer. Threshold small values to zero (δ_S), quantize (δ_Q), reconstruct with g fitted to the dense x. g adapts to the combined error and the backward pass sees both. For ternary sparse weights linear g is used, which keeps zeros at zero.

Q22. What is 1.5-bit quantization?

Answer. Three levels $\{-1, 0, 1\}$, which need $\log_2 3 \approx 1.58$ bits. At A1.5W1.5 STE gave NaN, g reached 0.3297.

Q23. How do you get sub-1 bit per element?

Answer. 1-bit values with 1:4 sparsity: one value plus a 2-bit position index per group of 4, so $(1 + 2)/4 = 0.75$ bits. This ignores the per-block scale and shift, which the paper does not count.

Q24. What is the energy proxy and how much should we trust it?

Answer. Sparsity factor $\times$ activation bits $\times$ weight bits $\times$ ops. BF16 is $16 \times 16 = 256$, A1W1 is 1. It is an estimate: no hardware measurement, no memory-traffic or overhead model.

Q25. How does this compare with LSQ?

Answer. LSQ (Esser et al., 2019) learns the step size by gradient descent, and its step-size gradient contains δ. It does not change the upstream gradient to x. The new method computes scale and shift in closed form and its Jacobian with respect to x contains δ.

Q26. Are s_f, b_f really constants?

Answer. No, they depend on $x_{\max}, x_{\min}$, and autograd flows through them. That only affects the two extreme entries per block. The paper's text and our equations hold them fixed; our Jacobian check excludes those entries.

Q27. Why did STE's activations blow up in your 1-bit run?

Answer. At 1 bit STE reconstructs to $x_{\min}$ or $x_{\max}$, which overshoots most entries. Least squares puts the two levels near the group means. Over many layers STE's residual stream grew to about $340\times$ g's.

Q28. Why didn't you reproduce the paper's loss explosion?

Answer. Less compute: batch 32×128 and 1000–2000 steps versus 64×256 and 5000. We did reproduce the stall, the activation blow-up, and an upward loss turn at $3\times$ learning rate.

Q29. What is Tiny Shakespeare and what do the loss numbers mean?

Answer. About 1.1M characters of Shakespeare dialogue; the model predicts the next character from a 65-character vocabulary. Loss in nats per character: random is $\ln 65 \approx 4.17$; STE stalls near 2.6; g reaches about 2.3; a float model reaches about 1.59.

Q30. Is the comparison in the paper fair?

Answer. The STE vs. g comparison is clean: same model, recipe and quantizer, only the reconstruction differs. Weaker points: single runs, no error bars, and BitNet run at A1W1 although its design uses 8-bit activations.

Q31. What is the main takeaway?

Answer. A good forward approximation can still give a bad gradient. The rounding error looks ignorable, but training needs information about it. Fitting the reconstruction to the data puts that information into the backward pass.